Kevin Vo - A16215024
10/3/2026
WES 237C

Project 2: Cordic and Phase Detector

# Project Goal

The goal of this project is to design and implement a Cartesian to Polar transform using two different algorithms: a COrdinate Rotation DIgital Computer (CORDIC) and through a lookup table (LUT). After a baseline is created and it is optimized, it will be compiled to run on an FPGA in the demo section. After that is done, in Project 2.5, I will take the FIR filter and the Phase Detector and combine it to accomplish a Phase Detector, which is a common block used in a digital communications receiver used to do simple synchronization and discover the phase of the signal.

# I. Question 1

**Question 1: One important design parameter is the number of rotations. Change that number to numbers between 10 and 20. This question should use a floating point implementation of CORDIC.**

**a) Create a table that shows resource usage, throughput, latency, and RMSE for each design you create. Use 10, 12, 14, 16, 18, and 20 rotations. You will need to add additional values to the table for 18 and 20 rotations. Chapter 3 has enough information to help you derive the additional angles and Kvalues.**
Using what I learned in Chapter 3, I determined the remaining Kvalues and angles for rotations index 16, 17, 18, 19, and 19 that were needed to answer Question 1.

| Iteration (i) [0 to 15 in starter code] | Kvalues ($2^{-i}$) | Angles (atan($2^{-i}$)) |
|---|---|---|
| **15 (Last one in the starter code)** | 3.05175781250000e-05 | 3.05175781155261e-05 |
| **16** | 1.52587890625e-05 | 1.52587890613e-05 |
| **17** | 7.62939453125-06 | 7.6293945311e-06 |
| **18** | 3.81469726563e-06 | 3.81469726561e-06 |
| **19** | 1.90734863281e-06 | 1.90734863281e-06 |

**Table 1.1:** Calculated values to fill in for index 16-20 of Kvalues and angles array.

**Table 1.2** displays the throughput, latency, RMSE of a CORDIC algorithm iterating through 10, 12, 14, 16, 18, and 20 rotations. The resource usage is quantified by measuring the amount of block ram (BRAM), 2 custom data paths (DSP), flip flops (FF), and lookup tables (LUT) used in each CORDIC algorithm design.

| Cordic 1A | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Rotations** | **RMSE (R)** | **RMSE (theta)** | **Est. Clock (ns)** | **Latency (Cycles)** | **Throughput (MHz)** | **BRAM** | **DSP** | **FF** | **LUT** |
| 10 | 0.000000799828342 | 0.001373887993 | 7.256 | 110 | 1.252881628 | 0 | 10 | 1357 | 2259 |
| 12 | 0.000000064680641 | 0.000274921942 | 7.256 | 130 | 1.060130608 | 0 | 10 | 1357 | 2261 |
| 14 | 0.00000012914046 | 0.00008095744852 | 7.256 | 150 | 0.9187798603 | 0 | 10 | 1357 | 2263 |
| 16 | 0.00000012914046 | 0.00001666481876 | 7.256 | 170 | 0.8106881121 | 0 | 10 | 1357 | 2265 |
| 18 | 0.000009679565665 | 0.000004742075362 | 7.256 | 190 | 0.7253525213 | 0 | 10 | 1358 | 2267 |
| 20 | 0.00001708450691 | 0.000006335243143 | 7.256 | 210 | 0.6562713288 | 0 | 10 | 1358 | 2269 |

**Table 1.2:** Measured values for latency, RMSE, throughput and resource usage of CORDIC 1A.

**b) Plot throughput, resource usage, and RMSE (theta and r on the same plot) as a function of the number of rotations. Clearly label your axes and each datapoint.** Figure 1.1, 1.2, and 1.3 all visualize the data in **Table 1.2** in bar graphs to easily compare and deduce trends in the data from.

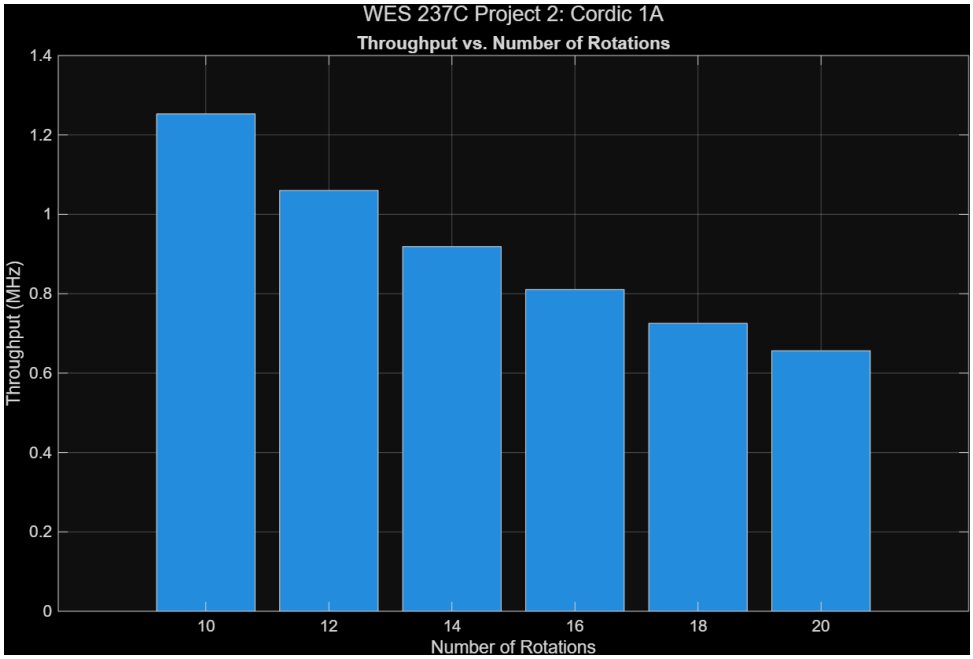



**Figure 1.1:** Bar graph comparing the calculated throughput in MHz of each CORDIC design rotations 10, 12, 14, 16, 18, and 20. It seems throughput decreases as you increase rotations.

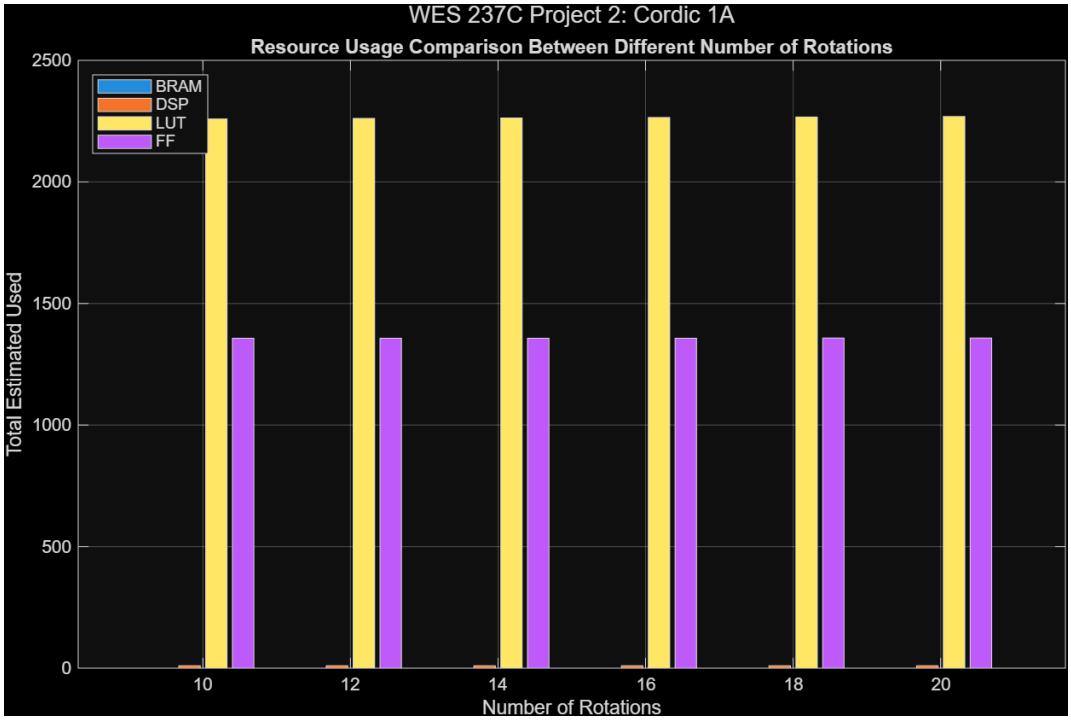



**Figure 1.2:** Bar graph displaying the measured resource usage of each CORDIC design with rotations 10, 12, 14, 16, 18, and 20. It seems increasing the number of rotations has no impact on resource utilization.

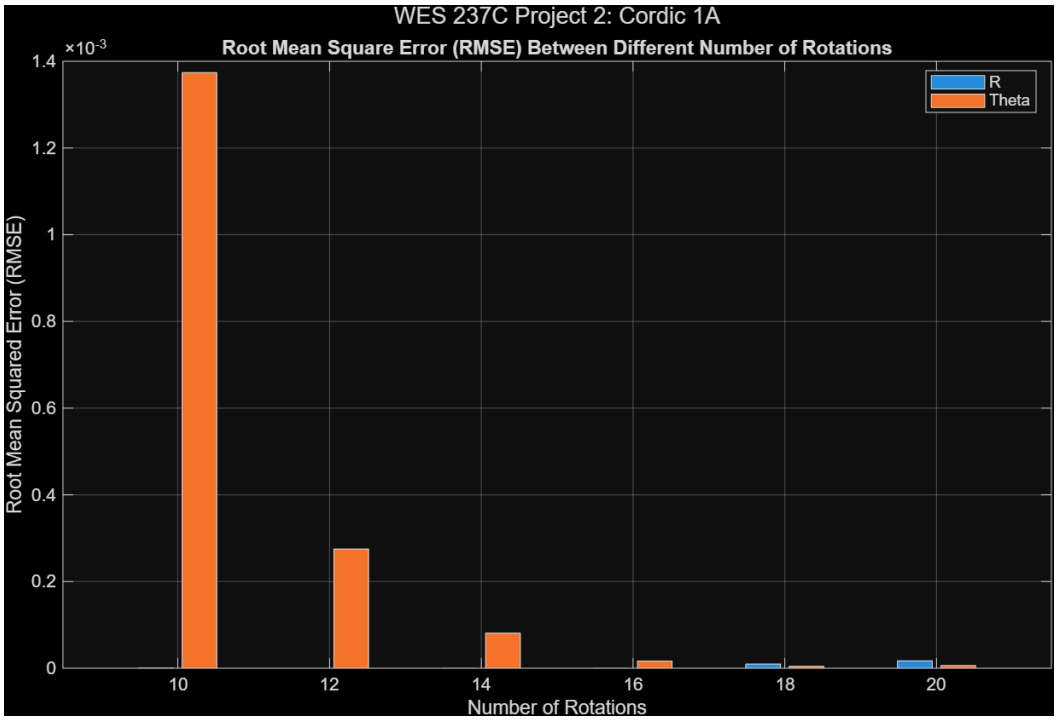



**Figure 1.3:** Bar graph displaying the measured root mean square error (RMSE) of r and theta values for each CORDIC design with rotations 10, 12, 14, 16, 18, and 20. According to the data the number of rotations plays an important role in decreasing the RMSE of both r and theta values.

**c) At what number of rotations does the accuracy stop noticeably improving in the plot?**
According to FIgure 1.3, it tells me that the accuracy stops noticeably past 16 rotations since that is when the RMSE decrease is negligible.

---

# II. Question 2

**Question 2: Another important design parameter is the data type of the variables.**
**a) We will use the ap_fixed arbitrary precision data type for each variable. At least how many integer bits are required for each variable? Remember that this is a signed type. (Hint: consider the range of values that each variable can take on. You can use the float implementation to help you determine this. Think of the range of values of the variables r, x, y, and theta). Give an answer for each variable. The testbench assumes that x and y are normalized between [-1, 1].**

The variables x, y, r, and theta are all the main variables of consideration in this question. Since I want to be consistent, I am keeping fixing the amount of rotations to be the default 16 as defined in the starter script. To determine the amount of integer bits required for each variable, we need to explore what possible ranges that r, x, y, and theta will take up.

According to the question, x and y are assumed to be normalized between -1 and 1. However, in the CORDIC operation, according to the textbook on Chapter 3, every rotation will scale the magnitude of the vector by a certain factor as defined by *Equation 3.26*:

$$A = \frac{1}{K} = \lim_{n\to\infty} \prod_{i=0}^{n-1} \sqrt{1 + 2^{-2i}} \approx 1.64676025812107$$

This equation tells us that as the number of iterations increases, the maximum possible value it can have stabilizes around 1.64676025812107. Since this is a scaling factor, this means that it will be multiplied by -1 and 1, which means that the possible ranges that x and y can take up are +/-1.64676025812107. To make sure that the whole range is encompassed, and using the two's complement calculation ($-2^{2-1}$ = -2 to $2^{2-1}$ = 2), we can round the range up to [-2, 2] to accommodate all possible values within the possible range of x and y of [-2, 2]. This range will require at least 1 integer and 1 sign bit to represent for a total of 2 bits.

Using the same logic we can find the range for r, the magnitude. The equation for magnitude is $r = \sqrt{x^2 + y^2}$ and enumerated equation accounted for the max and minimum range being $r = \sqrt{1^2 + 1^2} = 1.41421356237$. As such, we need 2 bits to represent all values within this range with 1 integer and 1 sign bit.

Lastly, the value of theta will fall in the range of [-pi, pi] (-3.14, 3.14) which represents the radius of the unit circle. Using similar logic to the previous formulas, we will need a round up the range

to accommodate the full range of pi, so the range of [-4, 4] is needed as determined using the two's complement formula $-2^{3-1} = -4$ to $2^{3-1} = 4$.

**Table 2.1** summarizes my calculations and the bit ranges.

| Variable | Possible Range | Bit Calculation | Adjusted Range | Int Bits | Signed Bits | Total Bits |
|---|---|---|---|---|---|---|
| **x, y** | [-1.64676, 1.64676] | $-2^{2-1} = -2$ to $2^{2-1} = 2$ | [-2,2] | 1 | 1 | 2 |
| **r** | $[-1.414, 1.414]$ | $-2^{2-1} = -2$ to $2^{2-1} = 2$ | [-2,2] | 1 | 1 | 2 |
| **theta** | [-3.14, 3.14] | $-2^{3-1} = -2$ to $2^{3-1} = 2$ | [-4,4] | 2 | 1 | **3** |

**Table 2.1:** Calculated ranges for variables x, y, r, and theta and their corresponding total integer (int) bits.

**b) Now that you have fixed the number of integer bits (use the largest number of integer bits determined in 2a), experiment with the number of total bits for each variable. Use the datatype for each variable. Create a table that shows resource usage, throughput, latency, and RMSE for each design you create. Create one plot each for resource utilization and RMSE vs total bits. Use 8, 12, 16, 20, 24, and 32 total bits.**

Since the theta variable requires an integer bit count of 3 bits, it is the largest number of bits needed and because of that we will be using 3 integer bits as the fixed number of integer bits that will be used in all of the future questions. Table 2.2 summarizes the resource usage, throughput, latency, and RMSE for each bitwidth for all variables and Figure 2.1, 2.2, and 2.3 visualizes this data.

| Cordic 2B | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Bits** | **RMSE (R)** | **RMSE (theta)** | **Est. Clock (ns)** | **Latency (Cycles)** | **Throughput (MHz)** | **BRAM** | **DSP** | **FF** | **LUT** |
| 8 | 0.05718297884 | 0.04792343825 | 6.492 | 36 | 4.278770453 | 0 | 0 | 135 | 486 |
| 12 | 0.006526475307 | 0.004206847399 | 6.154 | 37 | 4.391782097 | 0 | 3 | 208 | 448 |
| 16 | 0.0004247922334 | 0.0006779825781 | 6.792 | 37 | 3.979244262 | 0 | 3 | 278 | 584 |
| 20 | 0.0000340348015 | 0.00003129855031 | 6.814 | 37 | 3.966396687 | 0 | 3 | 346 | 696 |
| 24 | 0.000002336506668 | 0.00001701554356 | 6.918 | 37 | 3.906768868 | 0 | 6 | 413 | 868 |
| 32 | 0.000000035025234 | 0.00001659139161 | 7.16 | 54 | 2.586385268 | 0 | 12 | 1044 | 1112 |

**Table 2.2:** Measured values for latency, RMSE, throughput and resource usage of CORDIC 2B.

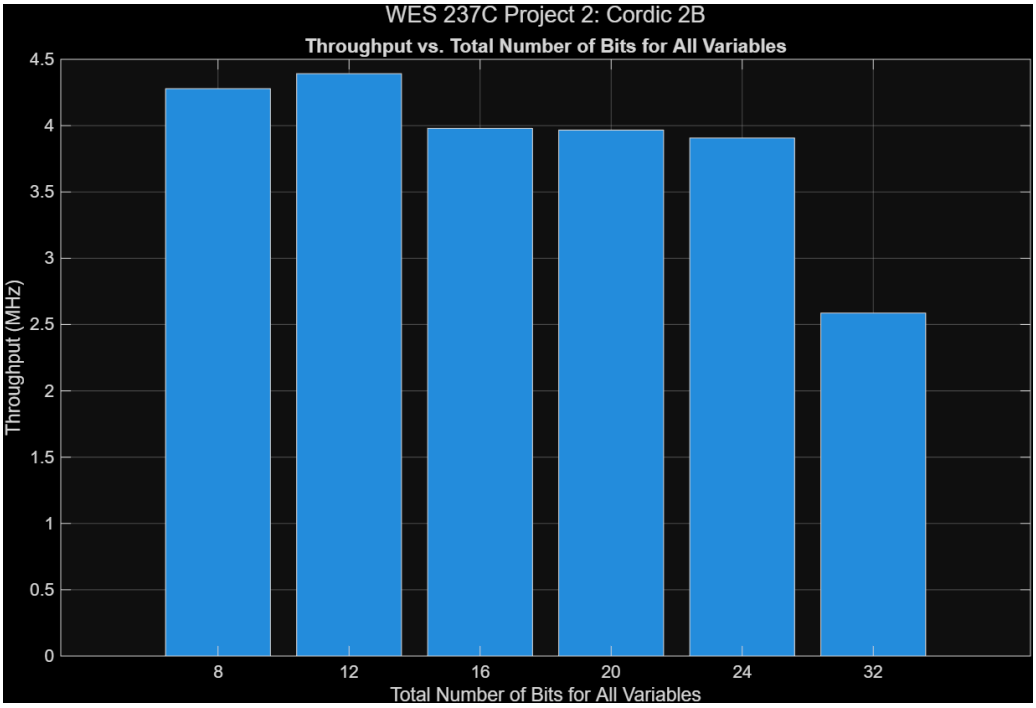



**Figure 2.1:** Bar graph comparing the calculated throughput in MHz of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for all variables. It seems throughput decreases as you increase the number of bits.

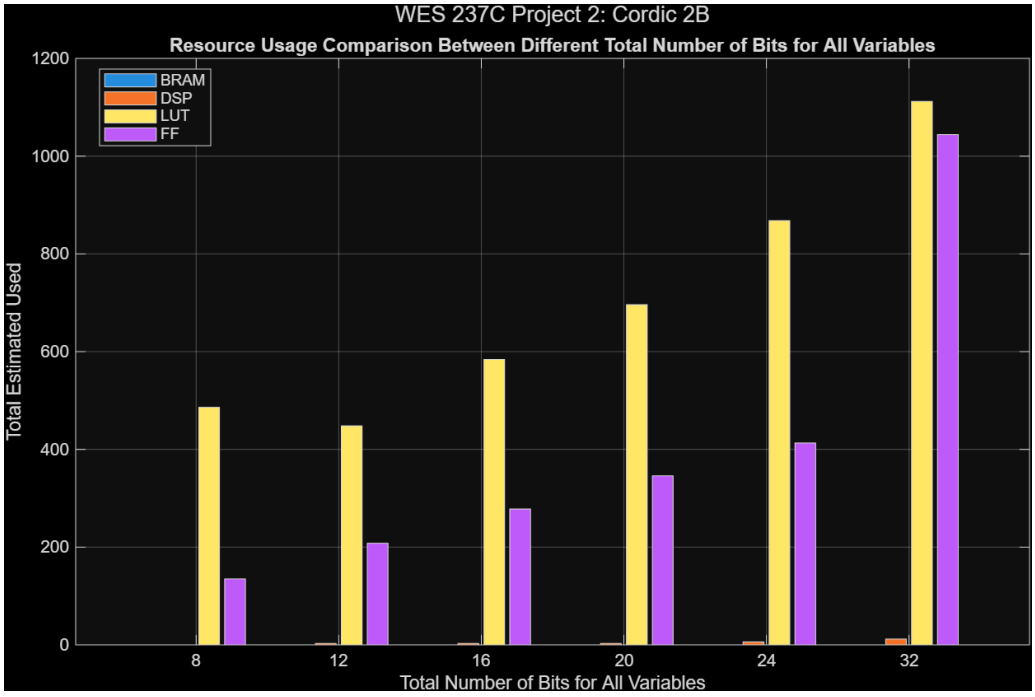



**Figure 2.2:** Bar graph displaying the measured resource usage of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for all variables. Resource usage unsurprisingly increases as the total number of bits increases.

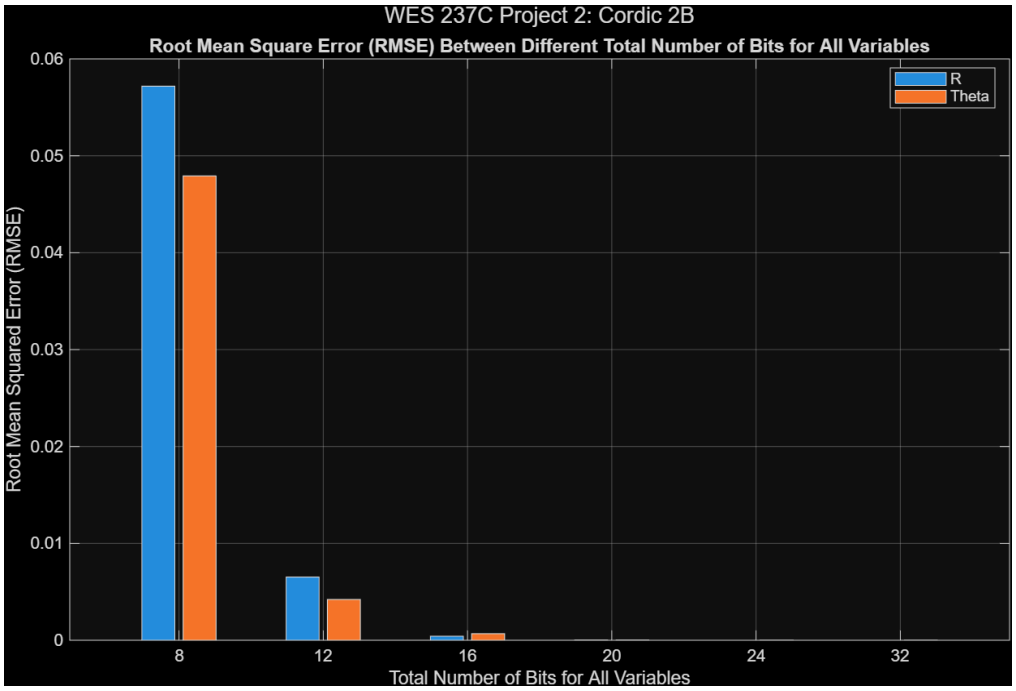



**Figure 2.3:** Bar graph displaying the measured root mean square error (RMSE) of r and theta values of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for all variables. According to the data the number of bits plays an important role in decreasing the RMSE of both r and theta values.

**c) Use ap_fixed<16,3> for all variables. Now experiment with changing the type of only the CORDIC rotation tables (Kvalues and angles). Create a table that shows resource usage, throughput, latency, and RMSE for each design you create. Use 4, 8, 12, 16, 20, and 32 total bits. Also plot RMSE (one plot) as a function of the total number of bits for the data type.**

| Cordic 2C | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Bits** | **RMSE (R)** | **RMSE (theta)** | **Est. Clock (ns)** | **Latency (Cycles)** | **Throughput (MHz)** | **BRAM** | **DSP** | **FF** | **LUT** |
| 4 | 0.05374278128 | 0.1859071404 | 6.792 | 37 | 3.979244262 | 0 | 1 | 218 | 530 |
| 8 | 0.0005806602421 | 0.02405916899 | 6.792 | 37 | 3.979244262 | 0 | 3 | 238 | 548 |
| 12 | 0.0004247922334 | 0.003003143705 | 6.792 | 37 | 3.979244262 | 0 | 3 | 258 | 566 |
| 16 | 0.0004247922334 | 0.0006779825781 | 6.792 | 37 | 3.979244262 | 0 | 3 | 278 | 584 |
| 20 | 0.0004247922334 | 0.001623038319 | 6.911 | 37 | 3.910725948 | 0 | 3 | 298 | 610 |
| 32 | 0.0004247922334 | 0.001623038319 | 7.267 | 53 | 2.596384275 | 0 | 5 | 595 | 785 |

**Table 2.3:** Measured values for latency, RMSE, throughput and resource usage of CORDIC 2C.

**Table 2.3** summarizes the resource usage, throughput, latency, and RMSE for iterating through the bitwidth for only the cordic rotation tables and Figure 2.4, 2.5, and 2.6 visualizes this data.

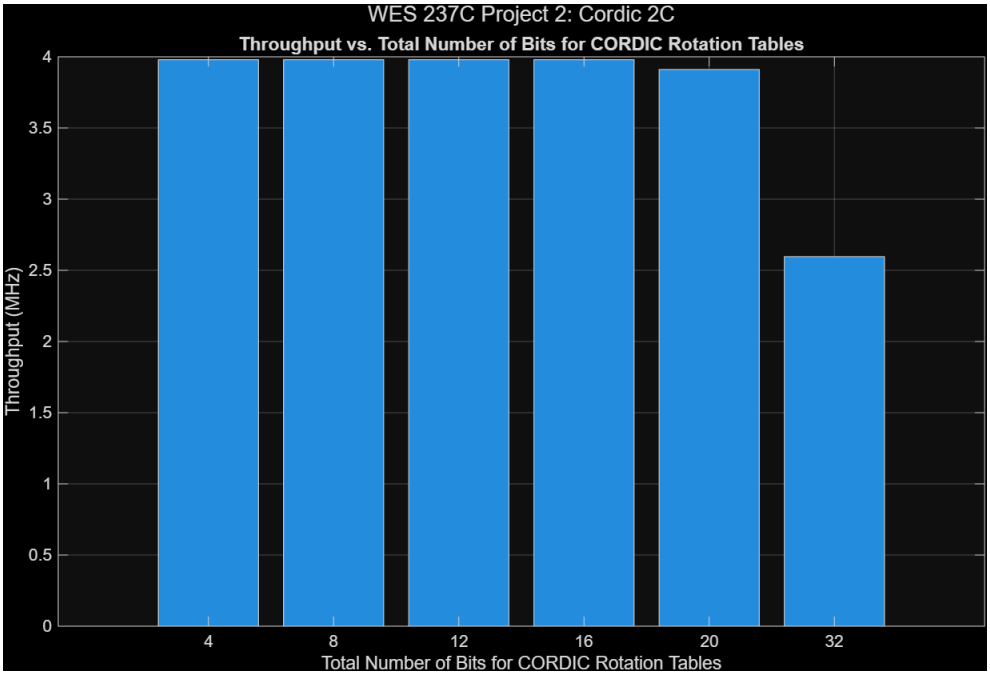



**Figure 2.4:** Bar graph comparing the calculated throughput in MHz of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for only cordic rotation tables. It seems throughput stays mostly the same as you increase the amount of bits.

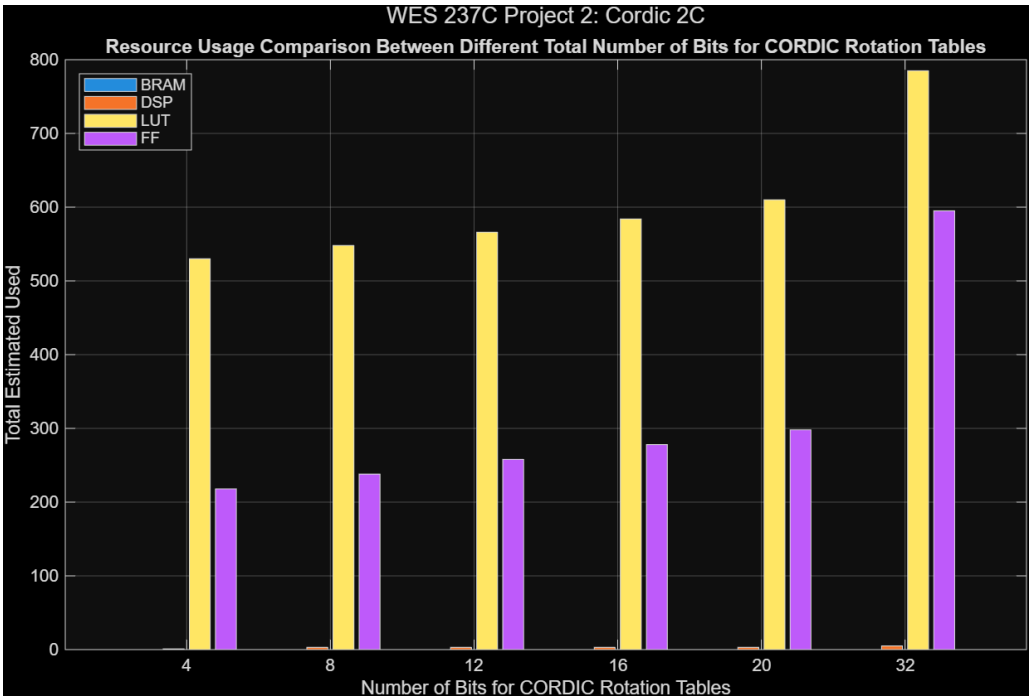



**Figure 2.5:** Bar graph displaying the measured resource usage of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for only cordic rotation tables. Resource usage unsurprisingly increases as the total number of bits increases.

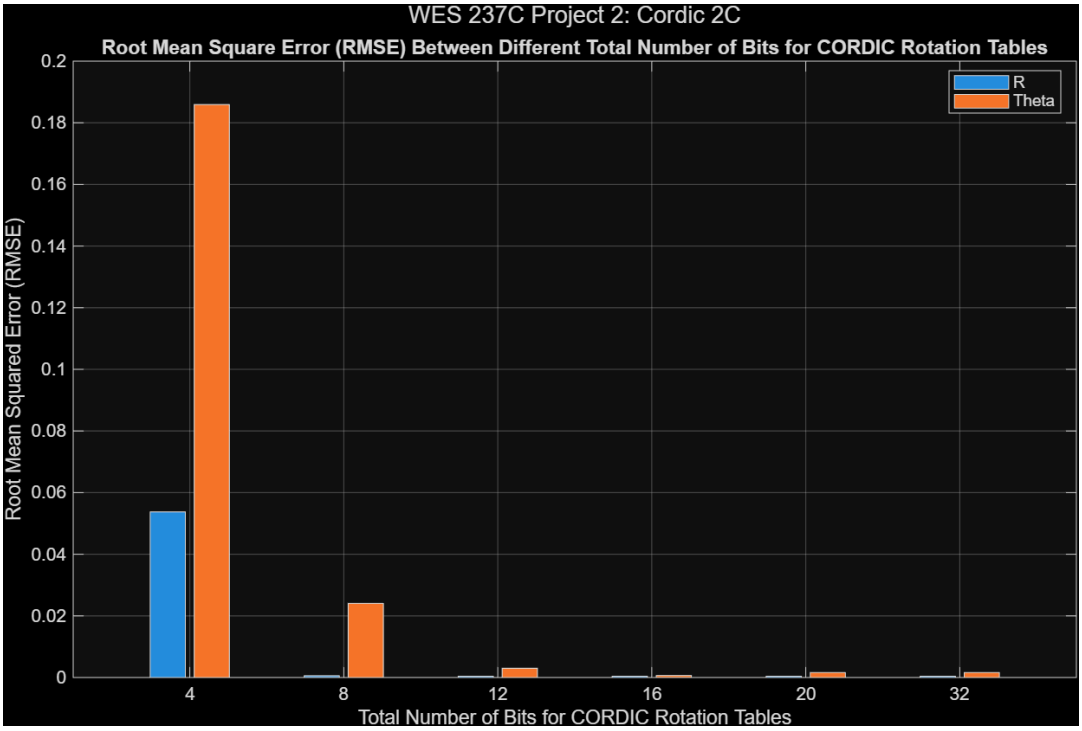



**Figure 2.3:** Bar graph displaying the measured root mean square error (RMSE) of r and theta values of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for only CORDIC rotation tables. According to the data the number of bits plays an important role in decreasing the RMSE of both r and theta values.

---

# III. Question 3

**Question 3: What is the effect of using simple operations (add and shift) in the CORDIC as opposed to multiply and divide?**

Using simple operations (add and shift) in CORDIC 3A as opposed to multiply and divide in Cordic 3A seems to increase the throughput and LUTs used a significant amount with using a 32-bit width compared to bit widths of 8, 12, 16, 20, and 24, where the throughput is similar. 3B also differs with 3A because no DSP is used, less FF are used (at 32-bit bit-width). These are some interesting observations that I observed in the plots below. Lastly, the root mean square error (RMSE) seems to decrease dramatically using simple operations than with multiply and divide. For instance on problem 3b the RMSE decreases to almost nothing for 12 bits of RMSE of r while theta shows the same thing.

**a) Now that you are using ap_fixed for all variables, change your implementation to use simple operations like add and shift instead of multiply and divide. Create a table that shows resource usage, throughput, latency, and RMSE for each bitwidth. Use 8, 12, 16,**

**20, 24, and 32 total bits. Use the implementation from 2b as a baseline for comparison. You may keep the final multiplication to account for gain.**

Table 3.1 summarizes the resource usage, throughput, latency, and RMSE for each bitwidth for all variables in a CORDIC with add shift operations and Figure 3.1, 3.2, and 3.3 visualizes this data.

| Cordic 3A | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Bits** | **RMSE (R)** | **RMSE (theta)** | **Est. Clock (ns)** | **Latency (Cycles)** | **Throughput (MHz)** | **BRAM** | **DSP** | **FF** | **LUT** |
| 8 | 0.1210008189 | 0.02476855367 | 7.073 | 37 | 3.821154677 | 0 | 0 | 132 | 459 |
| 12 | 0.0005685074138 | 0.003082329175 | 6.154 | 39 | 4.166562503 | 0 | 0 | 241 | 561 |
| 16 | 0.0003930192615 | 0.000541484158 | 6.792 | 38 | 3.874527308 | 0 | 0 | 325 | 751 |
| 20 | 0.0004542923416 | 0.0000114708937 | 6.814 | 38 | 3.862017827 | 0 | 0 | 413 | 903 |
| 24 | 0.0004629254981 | 0.00001701554356 | 6.918 | 38 | 3.803959161 | 0 | 0 | 500 | 1058 |
| 32 | 0.0004628919123 | 0.00001659139161 | 7.16 | 38 | 3.675389591 | 0 | 0 | 706 | 1458 |

**Table 3.1:** Measured values for latency, RMSE, throughput and resource usage of CORDIC 3A.

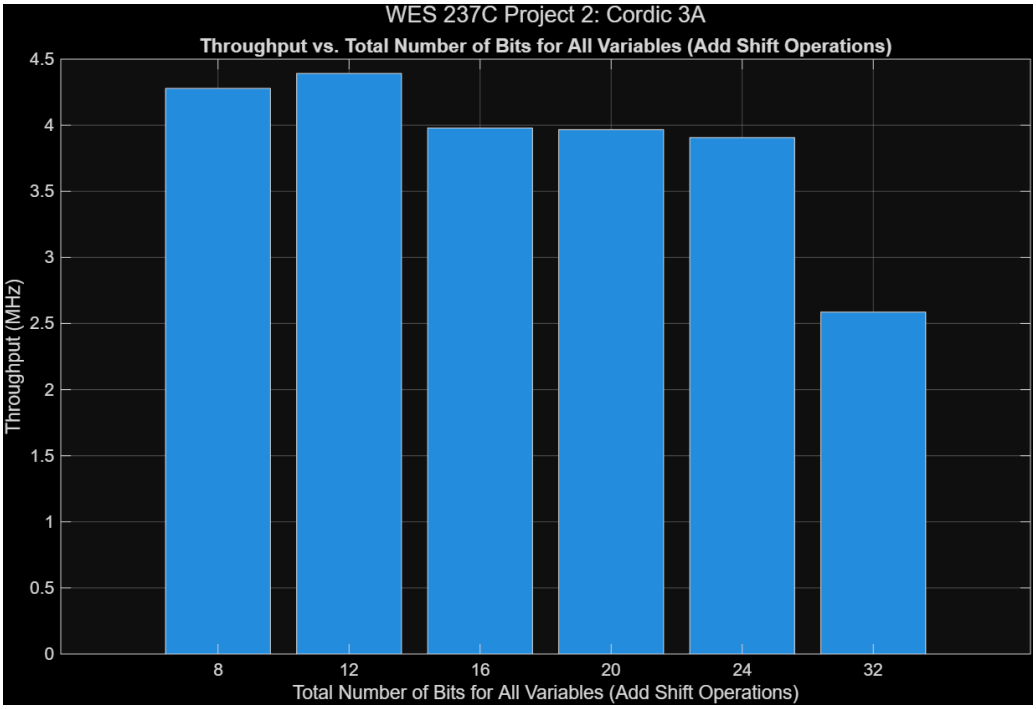



**Figure 3.1:** Bar graph comparing the calculated throughput in MHz of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for all variables in a CORDIC with add shift operations. It seems throughput stays mostly the same as you increase the amount of bits.

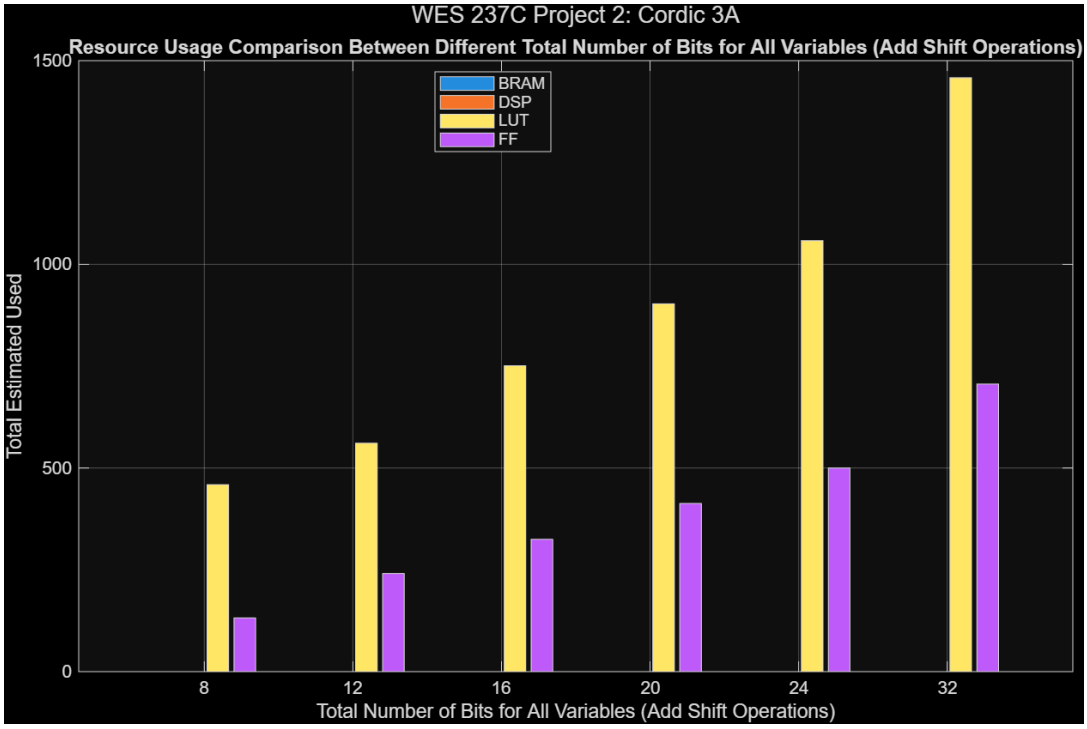



**Figure 3.2:** Bar graph displaying the measured resource usage of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for all variables in a CORDIC with add shift operations . Resource usage unsurprisingly increases as the total number of bits increases.

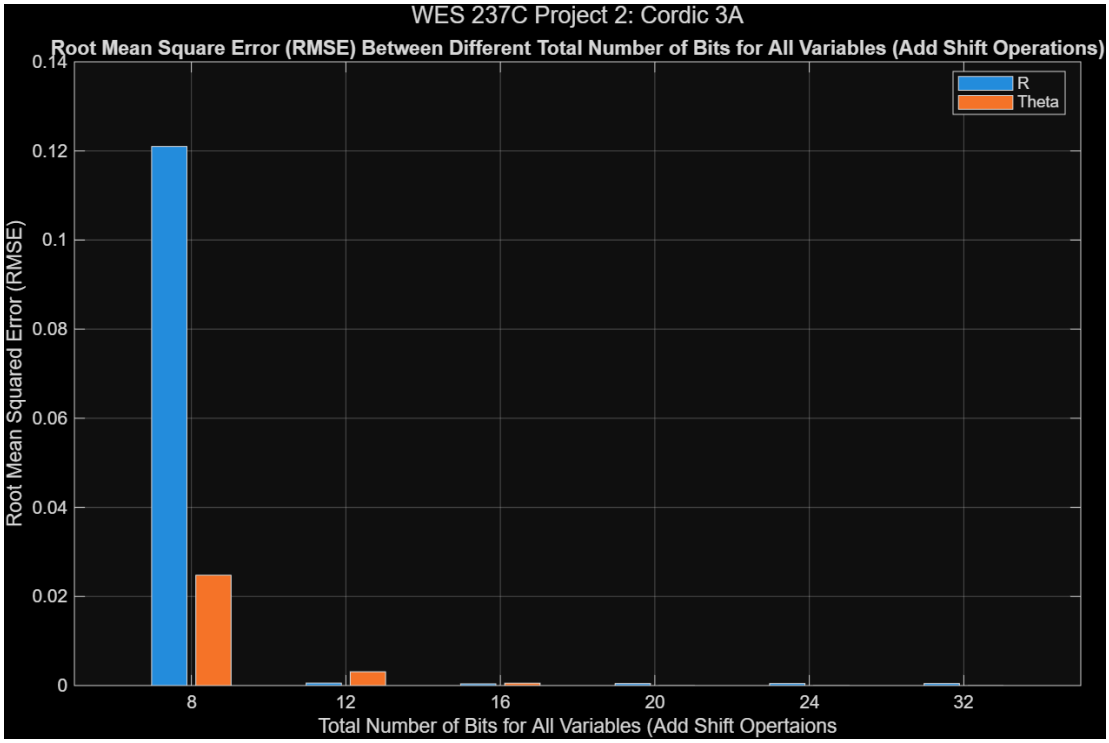



**Figure 3.3:** Bar graph displaying the measured root mean square error (RMSE) of r and theta values of each CORDIC design with 8, 12, 16, 20, 24, and 32 total number of bits for all variables in a CORDIC with add shift operations. According to the data the number of bits plays

an important role in decreasing the RMSE of both r and theta values. The error rate decreases significantly to almost 0 at around 12 bits and higher.

**b) Create 3 separate plots for LUTs, DSPs, and FFs for each of these data types and each implementation that compares these results to the results from 2b. Clearly label your axes and each datapoint. Use a different color/line style for each implementation.**

Figures 3.5, 3.6, and 3.6 compare the LUT, DSP, and FF between Cordic with multiplication and division and a Cordic with simple add and shift operations for all variables with bitwidths 8, 12, 16, 20, and 24 in a CORDIC in 2A with multiplication and divide operations vs. a CORDIC in 3B with add and shift operations. Figure 3.4, 3.7, and 3.8 were created to compare throughput, RSME r and RMSE theta respectively to keep it consistent.

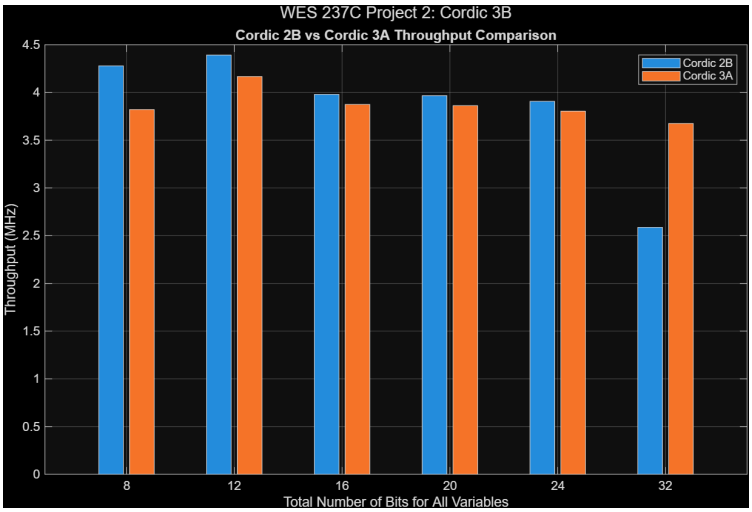



**Figure 3.4:** Bar graph comparing the calculated throughput in MHz for all variables with bitwidths 8, 12, 16, 20, and 24 in a CORDIC in 2A with multiplication and divide operations vs. a CORDIC in 3B with add and shift operations.

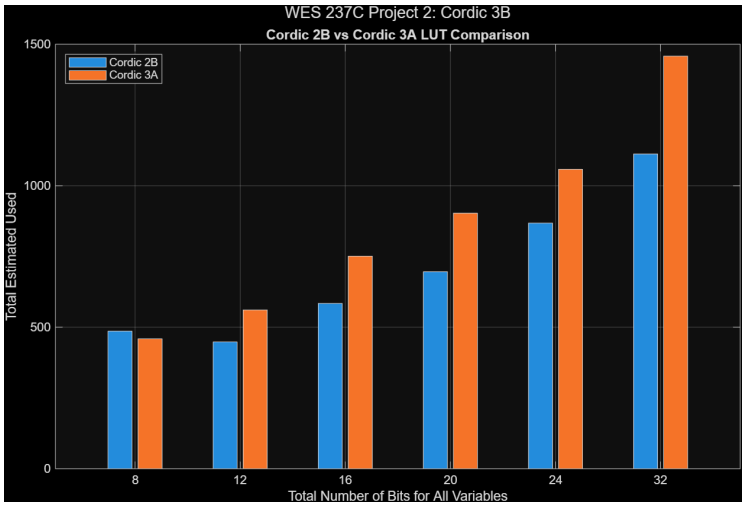



**Figure 3.5:** Bar graph comparing the measured LUT usage for all variables with bitwidths 8, 12, 16, 20, and 24 in a CORDIC in 2A with multiplication and divide operations vs. a CORDIC in 3B with add and shift operations.

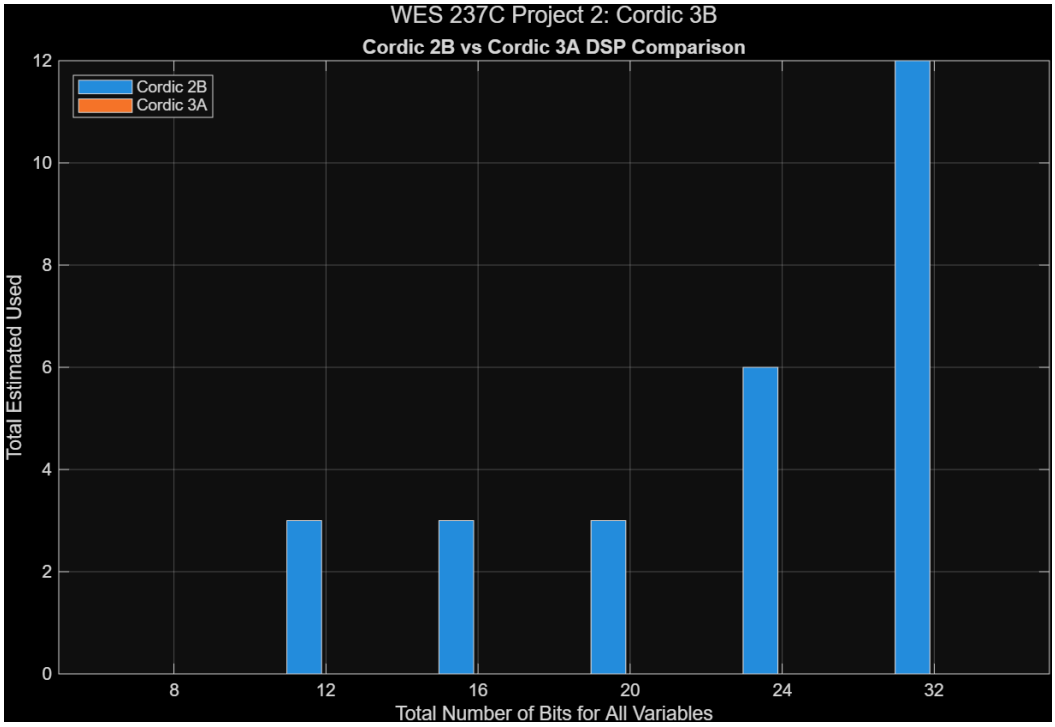



**Figure 3.6:** Bar graph comparing the measured DSP usage for all variables with bitwidths 8, 12, 16, 20, and 24 in a CORDIC in 2A with multiplication and divide operations vs. a CORDIC in 3B with add and shift operations.

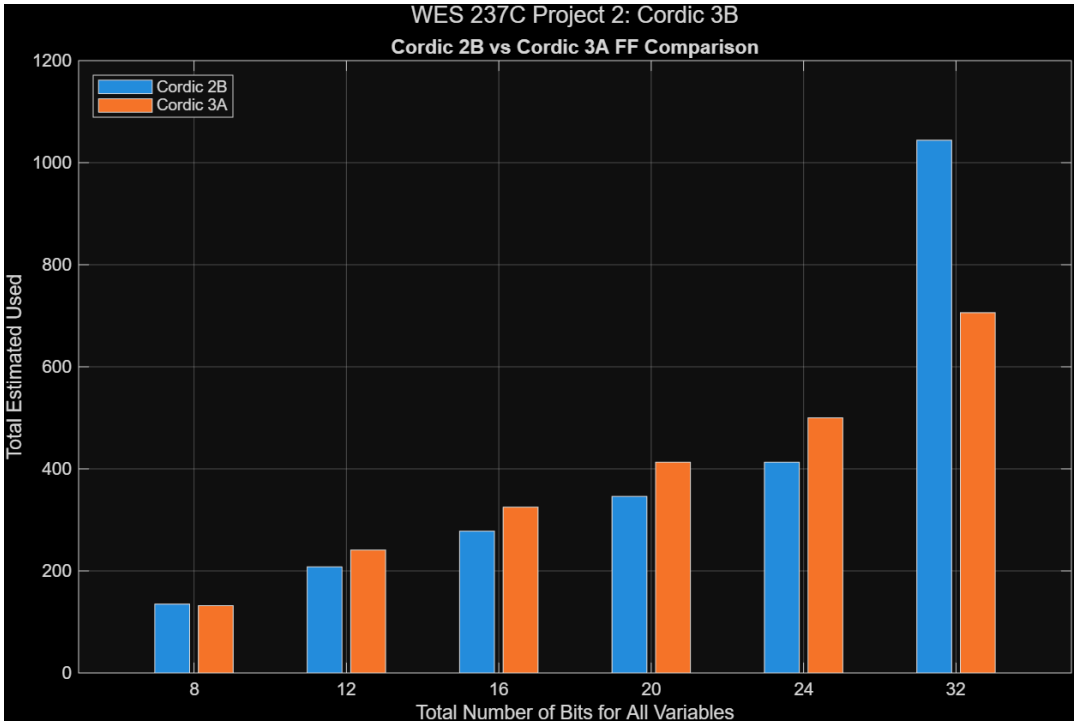



**Figure 3.7:** Bar graph comparing the measured FF usage for all variables with bitwidths 8, 12, 16, 20, and 24 in a CORDIC in 2A with multiplication and divide operations vs. a CORDIC in 3B with add and shift operations.

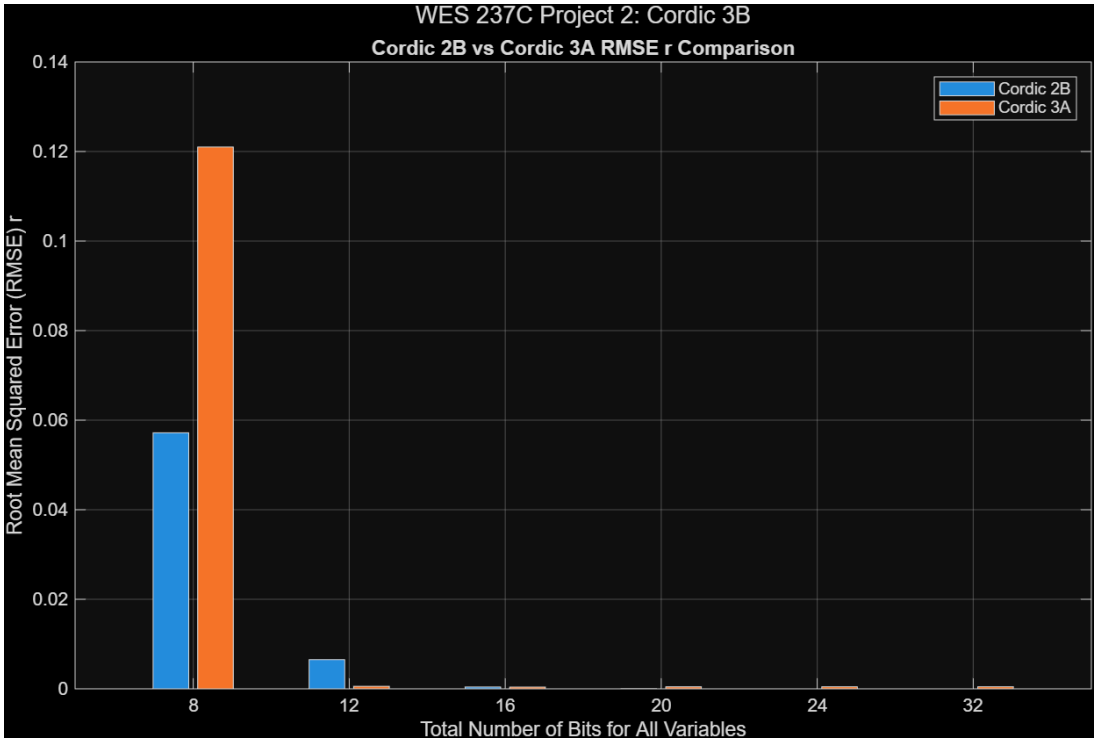



**Figure 3.8:** Bar graph comparing the measured RMSE of the variable r for all variables with bitwidths 8, 12, 16, 20, and 24 in a CORDIC in 2A with multiplication and divide operations vs. a CORDIC in 3B with add and shift operations.

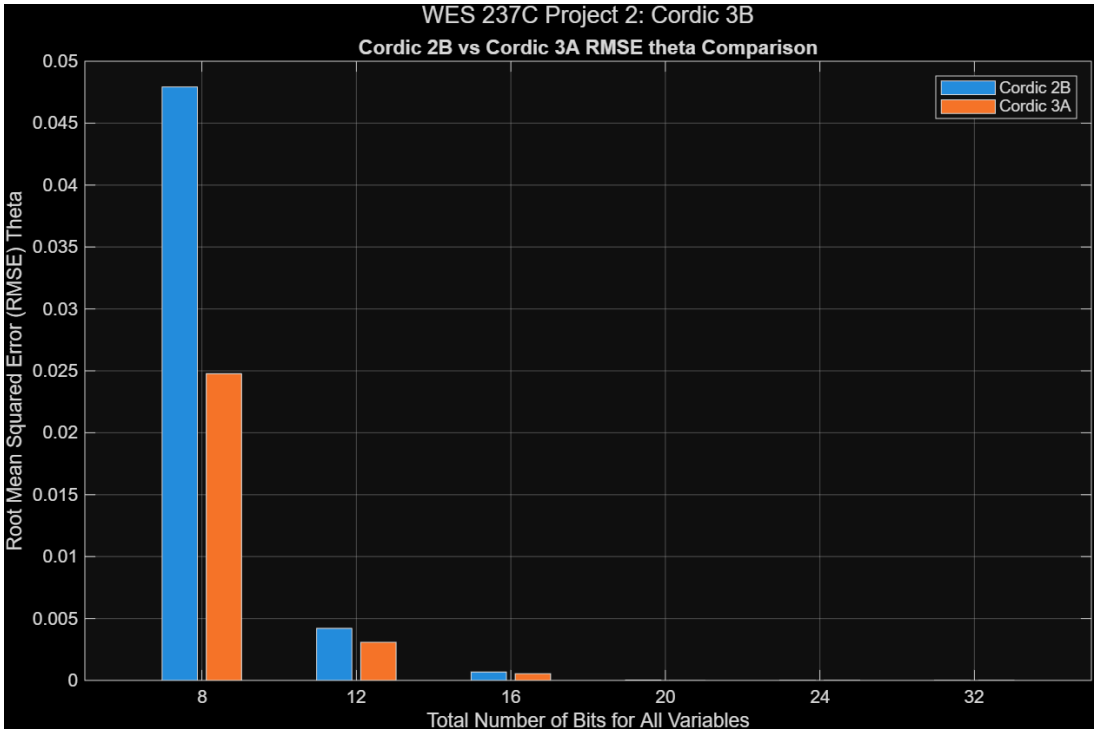



**Figure 3.9:** Bar graph comparing the measured RMSE of the variable theta for all variables with bitwidths 8, 12, 16, 20, and 24 in a CORDIC in 2A with multiplication and divide operations vs. a CORDIC in 3B with add and shift operations.

**Question 4: These questions all refer to the lookup table (LUT) implementation of the Cartesian to Polar transformation. This is in cordic_LUT/cordiccart2pol.cpp.**

**a) How does the input data type affect the size of the LUT? How does the output data type affect the size of the LUT? Precisely describe the relationship between input/output data types and the number of bits required for the LUT.**

For the LUT-based implementation, I saw that the HLS script expected a direct table lookup, so I used a python script to generate this lookup table. According to the cordiccart2pol.cpp script in the cordic_LUT folder, it requires an input lookup table of radius and theta values. The code as given initializes these values to be 0 and suggests using dummy values to get synthesis results. The python script generate_lut.py generates these values and I was able to get a successful test and a report generated for 4, 5, 6, 7, and 8 total bits.

**b) Create a table of resource usage, throughput, latency, and error vs number of total bits. Use the same number of integer bits for all data types (as in 2b). Use 4, 5, 6, 7, 8 total bits (8 bits will take some time to synthesize).**

Table 4.1 summarizes the resource usage, throughput, latency, and RMSE for each bitwidth for all variables in a CORDIC LUT implementation and Figure 4.1, 4.2, and 4.3 visualizes this data.

| Cordic LUT | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Bits** | **RMSE (R)** | **RMSE (theta)** | **Est. Clock (ns)** | **Latency (Cycles)** | **Throughput (MHz)** | **BRAM** | **DSP** | **FF** | **LUT** |
| 4 | 2.169364691 | 2.761842489 | 6.911 | 10 | 14.46968601 | 2 | 0 | 2 | 64 |
| 5 | 2.574248075 | 2.977366924 | 7.227 | 10 | 13.83700014 | 2 | 0 | 2 | 66 |
| 6 | 2.737981558 | 3.119124174 | 7.245 | 10 | 13.8026225 | 4 | 0 | 2 | 70 |
| 7 | 3.083806038 | 3.452608347 | 7.095 | 10 | 14.0944327 | 14 | 0 | 2 | 72 |
| 8 | 0.02309408411 | 0.05104573444 | 3.254 | 10 | 30.7314075 | 64 | 0 | 2 | 14 |

**Table 4.1:** Measured values for latency, RMSE, throughput and resource usage of CORDIC LUT.

**c) Plot all types of resource usage (LUTs, FFs, DSPs) as a function of the total number of bits for the data types. Make one plot for resource utilization**

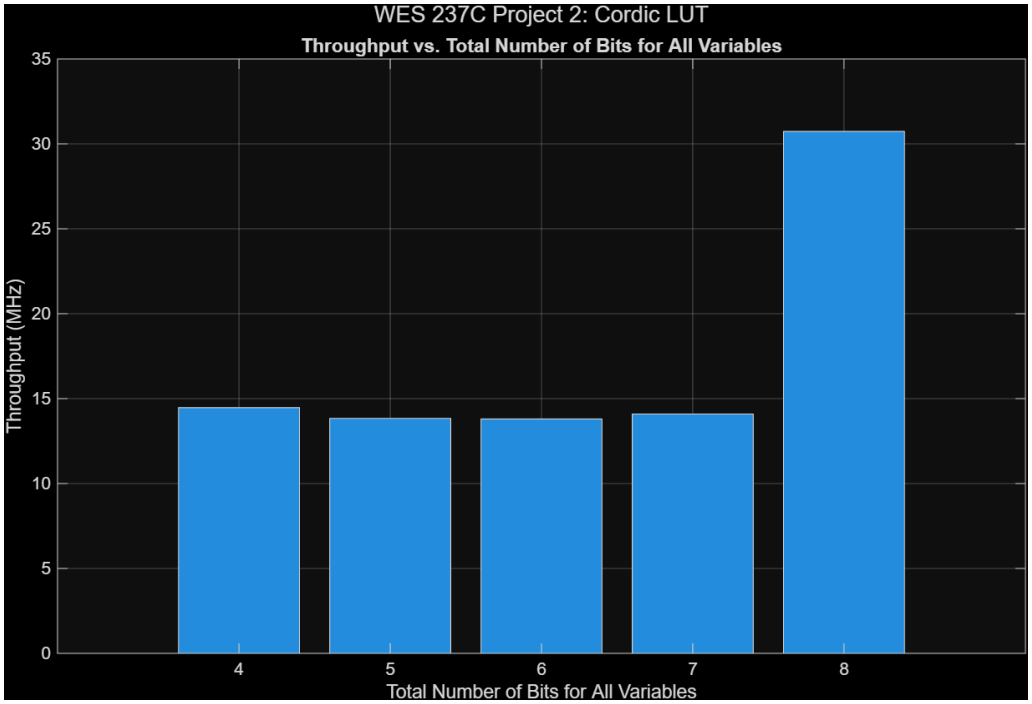



**Figure 3.1:** Bar graph comparing the calculated throughput in MHz of each CORDIC LUT implementation with 4, 5, 6, 7, and 8 total number of bits. It seems throughput stays mostly the same until the bitwidth is increased to 8, whereby there is a massive improvement in throughput from 15% to 30%.

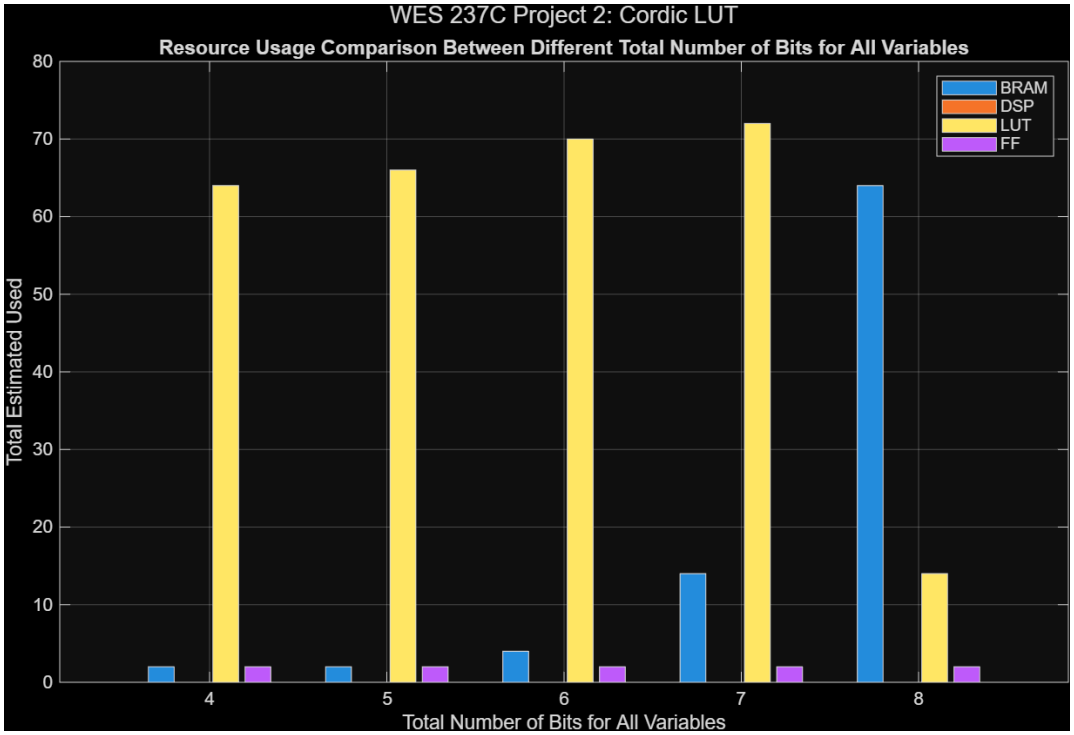



**Figure 3.2:** Bar graph displaying the measured resource usage of each CORDIC LUT implementation with 4, 5, 6, 7, and 8 total number of bits. Resource usage seem to mostly use LUT from 4-7, but switches to heavily in BRAM at total number of bits of 8 bits.

**d) Plot RMSE as a function of the total number of bits for the data types.**

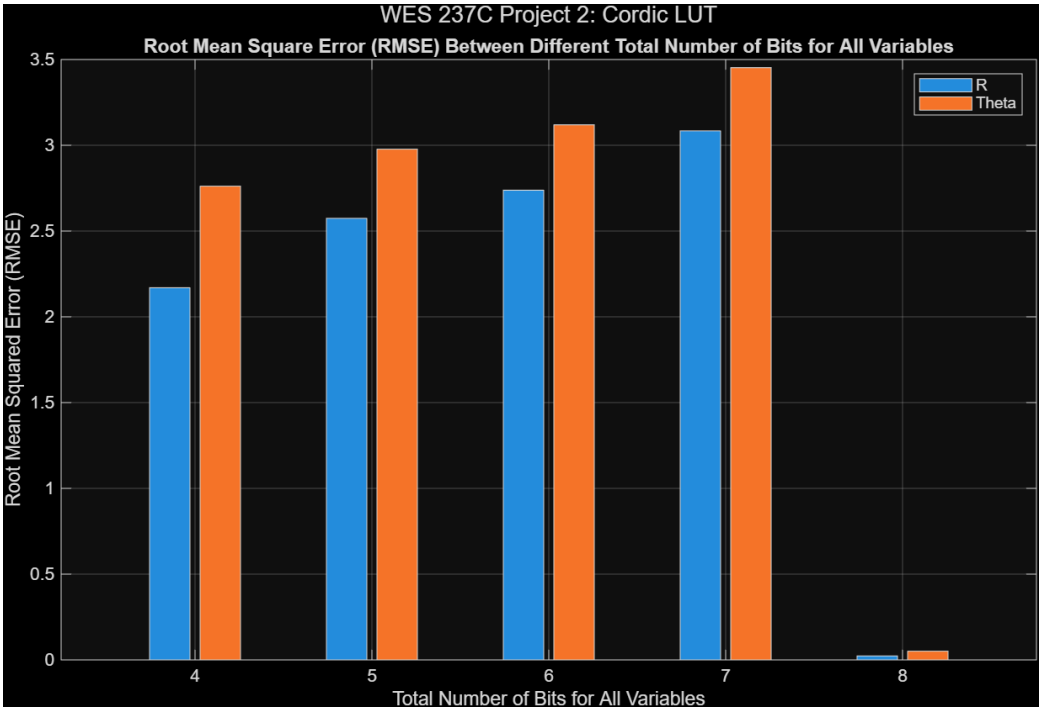



**Figure 3.3:** Bar graph displaying the measured root mean square error (RMSE) of r and theta values of each CORDIC LUT implementation with 4, 5, 6, 7, and 8 total number of bits. According to the data the number of bits plays an important role in decreasing the RMSE of both r and theta values.

**e) What advantages/disadvantages of the CORDIC implementation compared to the LUT-based implementation?**

Some advantages of the LUT-based implementation by far is the high amount of throughput and very low error compared to the CORDIC implementation. In Figure 3.1, 3.2, and 3.3 we see an 8-bit LUT implementation outperform all implementations in terms of throughput, resource usage, and low error rate. For context, the throughput of all the other implementations of Cordic only go up to a little less than 5%. For LUT implementation, I am getting 30% throughput.

---

# IV. Phase Detector

**Question 1: What is the throughput of your Phase Detector? How does that relate to the individual components (FIR, CORDIC, etc.)? How can you make it better?**

Did not get to this section.

# V. PYNQ Demo

Cordic PYNQ Board Demo: https://youtu.be/gtomDOQD-KM

# VI. Code Submission

The code for this project can be found through the link on my personal Github with username KVo5657: https://github.com/KVo5657/wes237c_submission/tree/main/cordic